MODULINK

Group 7 : Akila Hettigoda, Andre Chen, Zach Yu

ELEC_ENG 327 Final Project Report

Abstract **3**
Introduction **4**
Design Constraints and Requirements **5**
Engineering Standards **7**
Broader Considerations **7**
Design Description **9**
System Overview 9
Inter-Module Communication 9
PCBs 14
3D Printing (if applicable) 16
Final Product **16**
Initial Goal vs. Final Product 16
Performance and Limitations 19
Challenges Encountered 20
Planning and Organization 22
Market Research **24**
What you learned from interviews of experts 24
What you learned from interviews of non-expert 24
Conclusion **25**
What You Learned 25
What You Would Do Differently If You Could Start Over 25
Possible Next Steps (if you had more time) 25
References (if applicable) **26**
Appendices (if applicable) **26**
Class Feedback **26**

List of Figures

Figure	Caption	Page
1	Block diagram of communication pathways	9
2	Diagram of all nets shared between modules	10
3	Get pin pressed function.	11
4	keypadTask function for handling button presses	11
5	Code for connecting to Wi-Fi and serving webpage from SPIFFS	13
6	Code for handling WebSocket events from the webpage	14
7	Code for decoding rotary encoder signal patterns	14
8	Encoder signal patterns	15
9	Code for handling I2C reads	15
10	Code for initializing control lines	16
11	Macropad PCB	17
12	Encoder PCB	18
13	3D model of the macropad enclosure	19
14	Macropad and encoder board prototypes	21
15	Macropad V1	21
16	Encoder board V1	22
17	Encoder board V2 and macropad board V2 (final version)	23
18	Gantt chart	26
19	Complete BOM for macropad	26
20	Complete BOM for encoder module	27

Abstract

This report presents the development of Modulink, a modular productivity console. Modulink was inspired by a magnetic input system seen in a youtube video. The goal was to design a customizable input platform with a central macropad and magnetically connected peripheral modules, such as rotary encoders.

Development began with individual prototyping of both the macropad and encoder modules using breadboards and perfboards. After validating USB HID functionality, work progressed on building a WebSocket-based configuration server while awaiting custom PCB fabrication. Though initial exploration included Bluetooth connectivity, battery power, and custom magnetic connectors, time constraints led the team to prioritize the original minimum viable product (MVP): a wired macropad with up to nine hot-swappable modules using a custom GPIO protocol.

The final design supports real-time reconfiguration via a web interface, modular communication, and USB HID output, allowing users to map custom macros to any key. The magnetic modular system enables the connection of up to 9 peripheral modules, in addition to the central macropad, for a total of 10 functional units. These peripheral modules can be freely repositioned based on user preference, offering a flexible and ergonomic setup tailored to individual workflows.

Introduction

The goal of our project is to create a modular HID device allowing the user to better interact with their computer in an easily customizable way based on their use cases and needs. The system is centered around a primary module that incorporates a compact macropad in a 3x4 layout. Additional peripheral modules can be magnetically attached and electronically connected to the main module via integrated connectors, enabling users to expand and tailor the device according to specific productivity needs.

In our approach, we first determined the most important functionalities that our users were missing in their current HID devices. This was done through a combination of market research and interviews. From our research, we identified that keyboard shortcuts and dial-based adjustments were highly sought after. After identifying these functionalities, we organized and grouped them by themes. Then we began to look at the problem from a more technical point of view, identifying the necessary components to accomplish each of the requested functions, such as key switches and rotary encoders, and determining communication protocols to allow the modules to talk to each other and the computer. We then designed the supporting circuitry around the chosen components and wrote prototype code to test the core functionalities. The core functionalities that our first draft of code focused on were sensor readings and wifi connection/website interface. From these, we were able to get primitive, partial prototypes to confirm the viability of key functions and then design PCBs and more polished code. After two rounds of PCBs and many code revisions, we were able to get full functionality out of our design, and finally designed the enclosure for each module.

Design Constraints and Requirements

This project was developed with a clear set of functional requirements and practical constraints that guided the design from concept to prototype.

The key requirements included:

- **Wireless Communication**

  The device was required to operate wirelessly to eliminate the need for physical cabling and enable remote interaction.

  This requirement was fulfilled by incorporating Wi-Fi connectivity through the ESP32 microcontroller. Users can connect to the device via a local network to access and modify macro key configurations in real time through a browser-based interface.

- **Web Interface and Server-Side Scripting**

  A web-based interface was required to allow users to monitor and control the device. Additionally, server-side scripting was necessary to process user input and communicate data back to the hardware.

  This was implemented using a WebSocket communication protocol between the ESP32 and the web client. The use of WebSockets enabled low-latency, bidirectional communication, allowing users to send macro key combinations to the device and receive immediate status updates.

- **Sensor Integration**

  The system was required to include at least two types of sensors to provide interactive input capabilities.

  This requirement was met by integrating two rotary encoders, twelve mechanical keys

configured in a matrix layout, and six additional function buttons. These components served as input sensors, enabling physical interaction with the device and allowing for extensive user customization.

- **Real-Time Operating System (RTOS)**

  The system was required to run on a real-time operating system to support multitasking and ensure responsive behavior.

  FreeRTOS was used to manage concurrent tasks such as keypad scanning, encoder input handling, WebSocket communication, and USB Human Interface Device (HID) output. The RTOS ensured that all components of the system operated smoothly and efficiently without task interference.

In addition to these functional requirements, the project was shaped by two significant constraints:

- **Budget and Timeframe:**

  The total budget for the project was limited to $300, or $100 per group member. All hardware and development expenses had to fall within this limit. The design and prototyping process was also constrained to a 10-week timeline, from initial planning through to final demonstration.

- **Module Size and Alignment:**

  Based on market research highlighting the importance of preserving desk space, each module was restricted to a maximum footprint of 9 cm × 9 cm. To ensure compatibility and interchangeability, all modules shared the same dimensions. This uniformity also allowed the magnetic connectors used for attachment and communication to align precisely across modules.

Engineering Standards

I2C. In order to communicate between the different modules (namely to transfer input data from the peripheral modules to the central module), we used the I2C protocol (UM10204 published by NXP Semiconductors). In the final prototype, the central module acted as the I2C master, with all other modules acting as I2C slaves. I2C was also used to interface with the display.

We implemented the **USB HID (Human Interface Device) protocol** to enable the macropad to simulate keyboard inputs. By following the HID specification defined by USB.org, we ensured that each key press on the macropad generated a valid HID report containing modifier keys and keycodes. These reports were sent via the ESP32-S3's native USB HID functionality, allowing the device to emulate a standard keyboard. This approach provided cross-platform compatibility, required no additional drivers, and supported complex key combinations (e.g., Ctrl+Shift+D) reliably and efficiently.

Programming. To program each of the modules that we developed, we used the C++ language (ISO 14882:2011, aka. C++11) with the Arduino framework. The Arduino framework provided convenient library implementations of a lot of the functionality required for the project, including a GPIO library, USB device library, an HID keyboard library, and a WiFi-based web server.

Physical parameters. To interface between modules, our design requires usage of conventional 2.54 mm pitch spacing between pins.

Broader Considerations

As an HID device, the main concern regarding its broader impact is the device's ability to become a vector for potential hacking events. Currently, we do not have any intentional safeguards/security measures designed to prevent hacking; however, there are few coincidental features that make malicious attacks less likely. Firstly, the device interfaces with the rest of the world via a server that is hosted by the ESP and is set up in such a way that it is only easily accessible when on the same wifi network. This limits the potential geographical threat range to the user's near vicinity. Additionally, there is no DNS set up for the website, and one must know the exact IP address in order to access the configuration website. However, if one is able to gain both access to the local wifi network as well as the device IP address, they could use the device to send malicious HID commands to the user's computer. With the right HID commands, the user is at risk as many individuals keep important information on their computers.

In order to address this issue, it may be important to implement a website password to prevent unauthorized individuals from accessing the device configurations. Additionally, it may be useful to restrict certain commands from being configured and sent, keeping the user safe.

Design Description

System Overview

The system consists of a collection of modules working together to act as a single input device. There must be exactly one *central module* and a variable number of *peripheral modules*. The central module is responsible for actually establishing the connection to the user's computer, and conveys input data. The peripheral modules handle different user-facing input and output devices, and interface with the central module for coordination.

Inter-Module Communication

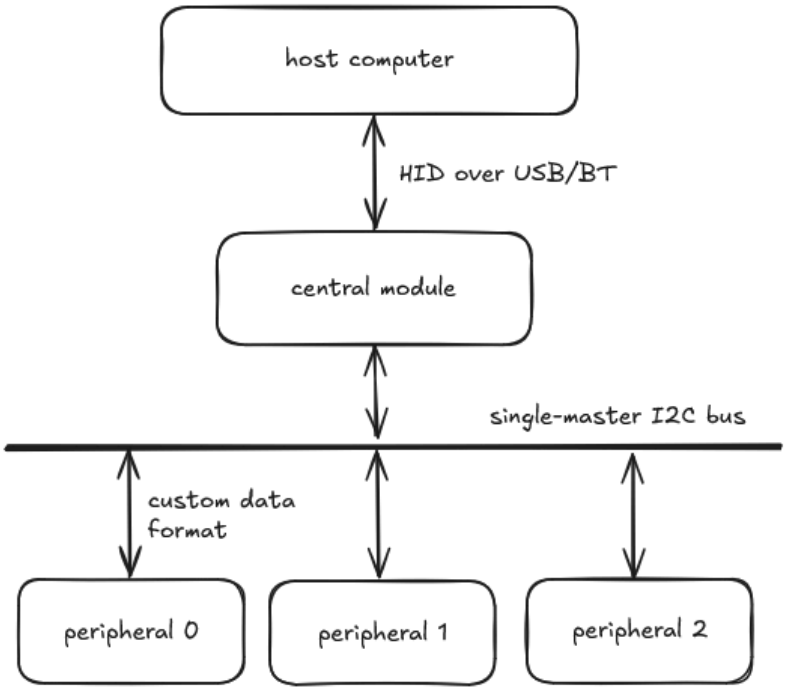

Figure 1: Block diagram of communication pathways.

The above block diagram shows the pathways in which information is transferred between modules and the host computer. All user input must go through the central module. If the user presses a button directly on the central module, it will be handled directly. If the user makes an input to the peripheral module, the information is processed first by the peripheral. The central module will be constantly polling the peripheral devices, and the peripheral device will reply with the updated information regarding the user's input over the I2C bus. Finally, the most up-to-date input information will be relayed to the host computer over USB in HID format.

All modules share common power lines and a shared I2C bus. There are 9 *liveness* lines used to detect when a peripheral module is connected, divided between the three connectors on the central module. Thus, each connector has 7 pins: 5V, GND, SDA, SCL, and 3 liveness lines.

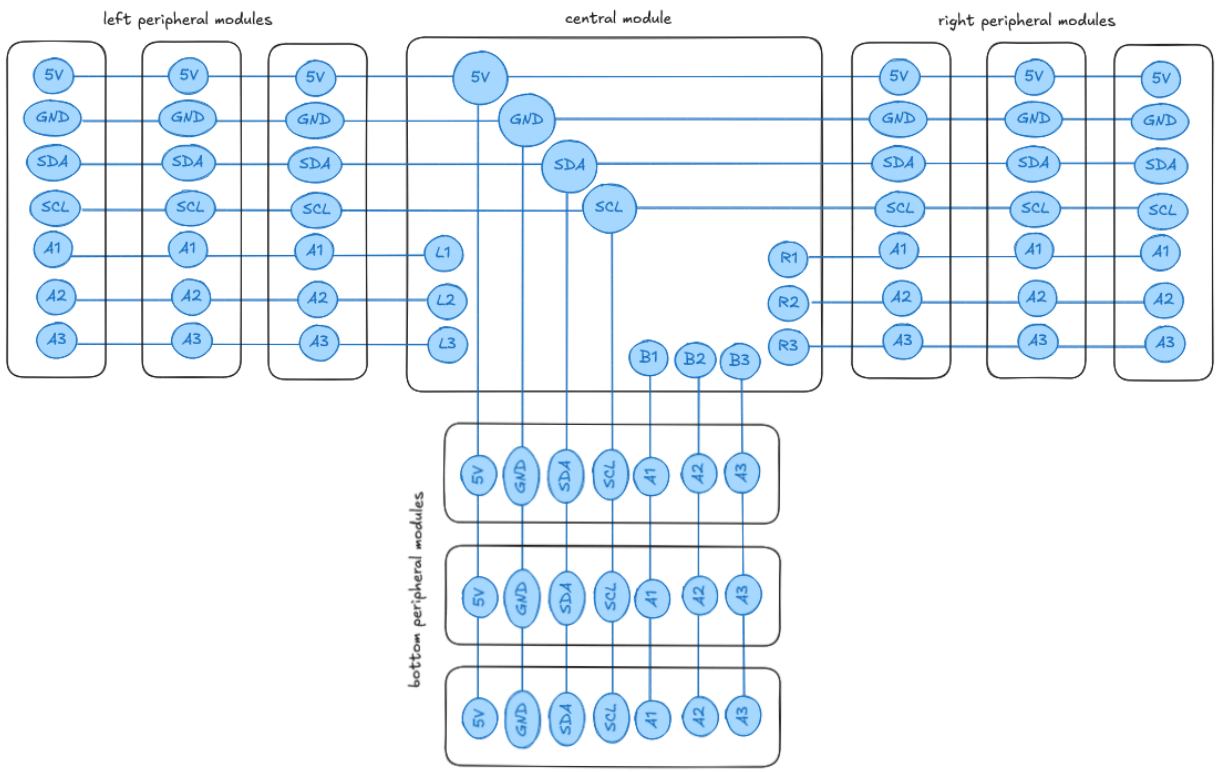

Figure 2: Diagram of all nets shared between modules.

Keyboard Scanning

To efficiently scan the 12 keys on the macropad while minimizing GPIO usage, we implemented a key matrix design with diodes to prevent ghosting. The matrix is arranged in rows and columns, where each column is sequentially pulled high while the row lines are read to detect any active key presses. This scanning process cycles rapidly through all columns, allowing us to reliably determine the state of each key with minimal latency and hardware overhead.

```
int getPressedButton() {
  static int lastButton = 0;
  static unsigned long lastDebounceTime = 0;
  const unsigned long debounceDelay = 30; // debounce time in ms

  int pressed = 0;

  for (int col = 0; col < 4; col++) {
    digitalWrite(colPins[col], HIGH);
    delayMicroseconds(100); // faster than delay(5) for responsive scan

    for (int row = 0; row < 3; row++) {
      if (digitalRead(rowPins[row]) == HIGH) {
        pressed = (row * 4 + col + 1);
        break;
      }
    }

    digitalWrite(colPins[col], LOW);
    if (pressed != 0) break;
  }

  if (pressed != lastButton) {
    lastDebounceTime = millis();
    lastButton = pressed;
    return 0; // wait for debounce
  }

  if (pressed != 0 && (millis() - lastDebounceTime) > debounceDelay) {
    return pressed;
  }

  return 0;
}
```

Figure 3: Get pin pressed function.

The macropad uses a FreeRTOS task to scan for key presses via the getPressedButton() function. When a valid key (1–12) is detected, the corresponding key combination—stored in the keyMappings array from the web configuration—is sent using the USB HID protocol. A short delay ensures each key is registered, and Keyboard.releaseAll() is called to reset the key state.

```
void keypadTask(void* parameter) {
  while (true) {
    int btn = getPressedButton();
    if (btn != 0) {
      // Handle switches 1-12 only
      if (btn >= 1 && btn <= 12) {
        Serial.printf("Switch %d pressed\n", btn);
        for (int i = 0; i < keyCounts[btn - 1]; i++) {
          Keyboard.press(keyMappings[btn - 1][i]);
          delay(10);
        }
        Keyboard.releaseAll();
      }
    }
    vTaskDelay(10 / portTICK_PERIOD_MS);
  }
}
```

Figure 4: keypadTask function for handling button presses

Configuration Webpage

The project utilizes SPIFFS to help us serve an HTML page. We chose SPIFFS as it was more convenient and cleaner than pasting HTML directly into the source code. The code initially connects to a hardcoded Wi-Fi network, and then hosts the webpage. The client-side webpage then establishes a WebSocket connection to the macropad for configuration purposes.

On the client side, the webpage displays a simple but beautiful interface for assigning shortcuts to keys. Each of the 12 keys of the macropad is associated with some text field describing its function. For example, the string “a” represents the A key, while the string “ctrl D” or “ctrl + D” represents the Ctrl + D keyboard shortcut. The user may edit these text fields to configure the function of each key. Additionally, the website listens for updates from the macropad over WebSocket about additional connected modules, and will display options for each new input. For example, when an encoder module (which has two rotary encoders and two keys) is connected, two dropdown options for encoder functionality and two more text fields for key functionality appear on the webpage; these are removed when the module is disconnected. The website includes a button to “Send Back” the new configuration to the macropad. This button parses each configuration option, checks for errors (which would halt the process and warn the user), converts the configuration into binary codes representing the actual input, and sends these binary codes as JSON over the WebSocket to the macropad.

This JSON format maps identifiers associated with each input to the binary codes for that input’s functionality. For example, {“key1”: “0xAB”} means that the first key on the macropad has been configured to send the 0xAB key on press.

```
void setup() {
  Serial.begin(115200);
  Serial.println("\nStarting...");

  setAllPixels(strip.Color(255, 0, 0)); // Red

  // Initialize SPIFFS
  if (!SPIFFS.begin(true)) {
    Serial.println("An Error has occurred while mounting SPIFFS");
    return;
  }
  Serial.println("SPIFFS mounted successfully");

  // List files in SPIFFS for debugging
  File root = SPIFFS.open("/");
  File file = root.openNextFile();
  Serial.println("Files in SPIFFS:");
  while(file) {
    Serial.print("FILE: ");
    Serial.println(file.name());
    file = root.openNextFile();
  }
```

```
#if defined(WIFI_NORMAL)
Serial.println("Connecting to WPA2-PSK...");
WiFi.begin(ssid, wifi_password);
#elif defined(WIFI_NU)
  Serial.println("Connecting to WPA2 Enterprise network");

  WiFi.disconnect(true);
  delay(1000);
  WiFi.mode(WIFI_STA);
  delay(500);

  esp_eap_client_set_identity((uint8_t*)eap_anonymous_id, strlen(eap_anonymous_id));
  esp_eap_client_set_username((uint8_t*)eap_id, strlen(eap_id));
  esp_eap_client_set_password((uint8_t*)eap_password, strlen(eap_password));
  esp_wifi_sta_enterprise_enable();

  WiFi.begin(ssid);
#endif

Serial.print("Connecting to WiFi");
display.println("Connecting to WiFi");
while (WiFi.status() != WL_CONNECTED) {
  delay(1000);
  Serial.print(".");
  display.print(".");
  display.display();
}
```

Figure 5: Code for connecting to Wi-Fi and serving webpage from SPIFFS

On the macropad side, the websocketevent() function manages all WebSocket communication between the macropad and the browser-based configuration interface. It listens for three types of events: client connection, disconnection, and incoming text data. When a client sends data (in JSON format), the function deserializes the payload and extracts user-defined configurations. For the 12 physical switches on the macropad, it reads and updates the keyMappings array with the new key combinations, also storing how many keys are assigned to each switch in keyCounts. For up to 9 additional modules, it configures each encoder's operating mode (encoder1_mode, encoder2_mode) and updates the key combinations assigned to their respective buttons.

```
// 6. Fix the websocketevent function to use modern ArduinoJson syntax
void websocketevent(byte num, WStype_t type, uint8_t * payload, size_t length) {
  switch (type) {
    case WStype_DISCONNECTED:
      Serial.println("Client Disconnected!");
      break;

    case WStype_CONNECTED:
      Serial.println("Client Connected");
      break;

    case WStype_TEXT:
      DeserializationError error = deserializeJson(doc_rx, (const char*)payload);
      if (error) {
        Serial.print("deserializeJson() failed: ");
        Serial.println(error.f_str());
        return;
      }
      // Parse and store switch key mappings (switches 1-12 only)
      for (int i = 1; i <= NUM_SWITCHES; i++) {
        String key = "switch" + String(i);
        if (doc_rx[key].is<JsonArray>()) {  // Modern syntax
          JsonArray keycodes = doc_rx[key].as<JsonArray>();
          keyCounts[i - 1] = keycodes.size();

          for (int j = 0; j < keycodes.size() && j < MAX_KEYS_PER_SWITCH; j++) {
            keyMappings[i - 1][j] = keycodes[j].as<uint8_t>();
          }
          // Debug print
          Serial.printf("Switch %d: ", i);
          for (int j = 0; j < keyCounts[i - 1]; j++) {
            Serial.printf("0x%02X ", keyMappings[i - 1][j]);
          }
          Serial.println();
        }
      }
      // Parse encoder configurations based on port numbers
      for (int port = 0; port < 9; port++) {
        // Parse encoder modes
        String encoder1Key = "encoder1_port" + String(port);
        String encoder2Key = "encoder2_port" + String(port);
        if (doc_rx[encoder1Key].is<const char*>()) {  // Modern syntax
          encoderArray[port].mode.encoder1_mode = doc_rx[encoder1Key].as<String>();
          Serial.printf("Port %d Encoder1 mode: %s\n", port, encoderArray[port].mode.encoder1_mode.c_str());
        }

        if (doc_rx[encoder2Key].is<const char*>()) {  // Modern syntax
          encoderArray[port].mode.encoder2_mode = doc_rx[encoder2Key].as<String>();
          Serial.printf("Port %d Encoder2 mode: %s\n", port, encoderArray[port].mode.encoder2_mode.c_str());
        }
        // Parse button key mappings
        String button1Key = "button1_port" + String(port);
        String button2Key = "button2_port" + String(port);

        // Parse button key mappings
        String button1Key = "button1_port" + String(port);
        String button2Key = "button2_port" + String(port);

        if (doc_rx[button1Key].is<JsonArray>()) {  // Modern syntax
          JsonArray keycodes = doc_rx[button1Key].as<JsonArray>();
          encoderArray[port].button_1_key_count = keycodes.size();

          for (int j = 0; j < keycodes.size() && j < MAX_KEYS_PER_SWITCH; j++) {
            encoderArray[port].button_1_keys[j] = keycodes[j].as<uint8_t>();
          }

          Serial.printf("Port %d Button1: ", port);
          for (int j = 0; j < encoderArray[port].button_1_key_count; j++) {
            Serial.printf("0x%02X ", encoderArray[port].button_1_keys[j]);
          }
          Serial.println();
        }

        if (doc_rx[button2Key].is<JsonArray>()) {  // Modern syntax
          JsonArray keycodes = doc_rx[button2Key].as<JsonArray>();
          encoderArray[port].button_2_key_count = keycodes.size();

          for (int j = 0; j < keycodes.size() && j < MAX_KEYS_PER_SWITCH; j++) {
            encoderArray[port].button_2_keys[j] = keycodes[j].as<uint8_t>();
          }

          Serial.printf("Port %d Button2: ", port);
          for (int j = 0; j < encoderArray[port].button_2_key_count; j++) {
            Serial.printf("0x%02X ", encoderArray[port].button_2_keys[j]);
          }
          Serial.println();
        }
      }

      break;
  }
}
```

Figure 6: Code for handling WebSocket events from the webpage

Encoder Board Code:

There are 3 notable components of the code of the encoder board firmware that will be discussed below: Sensor Reads, I2C reads, and Control Line Initializations.

```
void IRAM_ATTR readButton1(){
  if((currTime1 - prevTime1) < 100){
    prevTime1 = currTime1;
    return;
  }
  buttonValue1++;
  prevTime1 = currTime1;
  pressed1 = true;
}
```

```
void IRAM_ATTR readEncoder1() {
  int MSB1 = digitalRead(ENCODER_PIN_A1); // A
  int LSB1 = digitalRead(ENCODER_PIN_B1); // B


  int encoded1 = (MSB1 << 1) | LSB1;
  int sum1 = (lastEncoded1 << 2) | encoded1;


  // Adjusted logic so clockwise = +
  if (sum1 == 0b1101 || sum1 == 0b0100 || sum1 == 0b0010 || sum1 == 0b1011) encoderValue1++;
  if (sum1 == 0b1110 || sum1 == 0b0111 || sum1 == 0b0001 || sum1 == 0b1000) encoderValue1--;


  lastEncoded1 = encoded1;
  turned1 = true;
}
```

Figure 7: Code for decoding rotary encoder signal patterns

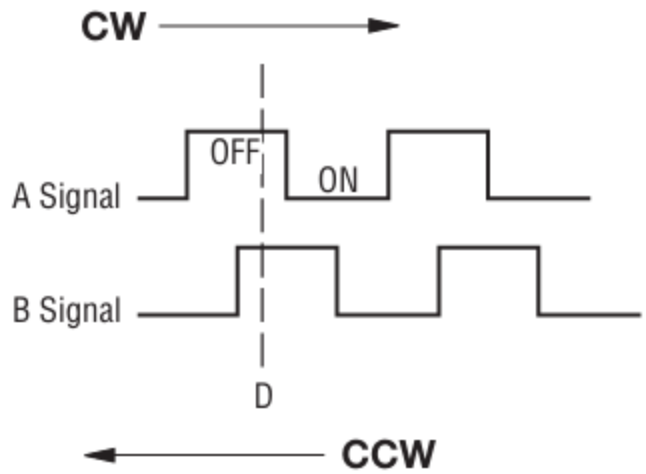

Figure 8: Encoder signal patterns

On the Encoder Board, there are 2 rotary encoders and 2 buttons. All 4 of these sensors are monitored by the MCU via interrupts. For the rotary encoders, the interrupts are set to trigger on CHANGE, as both rising and falling edges indicate important events. On each edge trigger, the digital value of the A and B lines from the encoder are written as least significant bits to a variable called sum. This variable keeps track of the current and past state of the encoder and is checked against the Encoder Direction Diagram to determine clockwise or counterclockwise spin. The buttons are much simpler, merely reacting on a falling edge, with an additional software based 100ms debounce filter.

```
void onI2cRead() {
  switch (readRegister) {
    case 1:
      Wire.write(0x97);
      break;
    case 2:
      Wire.write(encoderValue1);// ENCODER 1
      break;
    case 3:
      Wire.write(encoderValue2);// ENCODER 2
      break;
    case 4:
      Wire.write(buttonValue1);// BUTTON 1
      break;
    case 5:
      Wire.write(buttonValue2);// BUTTON 2
      break;
    default:
      Wire.write(0x99);
  }
}
```

```
void onInitialI2cRead(){
  switch(readRegister){
    case 0:
      Wire.write(encoder_address);
      break;

    default:
      Wire.write(readRegister);
      encoder_address = readRegister;
      readRegister = 0;
      break;
  }
}
```

Figure 9: Code for handling I2C reads

There are two different functions that get called when the macropad requests information from the encoder board: onInitialI2cRead and onI2cRead. Both methods use a large switch case that switches on a variable set by a code sent by the master module called readRegister. On startup, all peripheral devices have a default I2C address of 0x72, and the onInitialI2cRead method allows the mainboard to reassign the encoder board's I2C address as it sees fit. After the address is reassigned, the I2C peripheral is restarted with the new address, and the onI2cRead methods is used. Within the onI2cRead method, the 1 case sends a special code indicating the module type, differentiating it as an encoder over other potential peripheral device types. Cases 2 - 5 send the recorded value of each sensor, and the default case sends an error code indicating an incorrect request.

```
void initializeControlLine() {
  pinMode(LINE1, INPUT);
  pinMode(LINE2, INPUT);
  pinMode(LINE3, INPUT);

  int line1 = 1;
  int line2 = 1;
  int line3 = 1;
  bool occupied = true;
  while(occupied){
    line1 = digitalRead(LINE1);
    line2 = digitalRead(LINE2);
    line3 = digitalRead(LINE3);
    if((line1 == LOW) || (line2 == LOW) || (line3 == LOW)){
      occupied = false;
    }
    Serial.println("stuck in control");
  }
  Serial.println("");
  Serial.println("complete control");


  if (line1 == LOW) {
    pinMode(LINE1, OUTPUT);
    digitalWrite(LINE1, HIGH);
    Serial.println("Occupying line 1");
  } else if (line2 == LOW) {
    pinMode(LINE2, OUTPUT);
    digitalWrite(LINE2, HIGH);
    Serial.println("Occupying line 2");
  } else if (line3 == LOW) {
    pinMode(LINE3, OUTPUT);
    digitalWrite(LINE3, HIGH);
    Serial.println("Occupying line 3");
  } else {
    Serial.println("I fcked up.");
  }
```

Figure 10: Code for initializing control lines

The initializeControlLine method is responsible for managing the control line communication with the main board at startup. When a peripheral device is connected, the

method is called which first scans the 3 available control lines. If any of the lines are read as LOW , the peripheral device will pull the line HIGH, indicating to the master device that there is a device present. This will start the I2C handshake process described above, allowing the peripheral device to be used. If no lines are read as LOW, this indicates that all 3 positions are already filled, and the peripheral device must wait for a device to be disconnected before being initialized.

PCBs

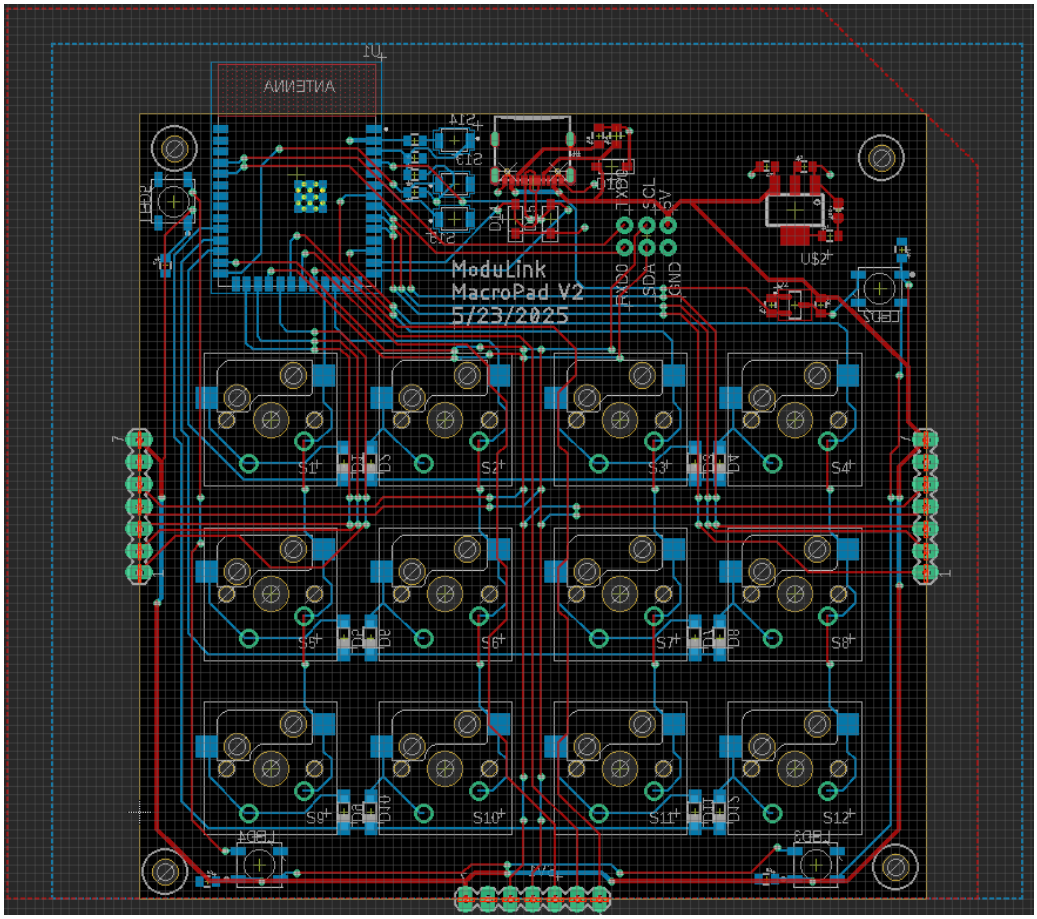

Figure 11: Macropad PCB

The above PCB is our macropad board, which features a few key design decisions. First of all, the ESP32S3 was positioned at the edge of the board such that the antenna hung off the edge

of the board. This placement prevents any PCB wiring from closely interfering with the signal coming from the antenna. Secondly, in an attempt to minimize the distance of the I2C communication lines, the 2 lines were routed to each connector from the center of the board. This equalizes the distance to all 3 connectors, and keeps the communication line lengths as short as possible to minimize bus capacitance. Finally, all 12 of the keys are equally spaced to make it as uniform and easy to use as possible.

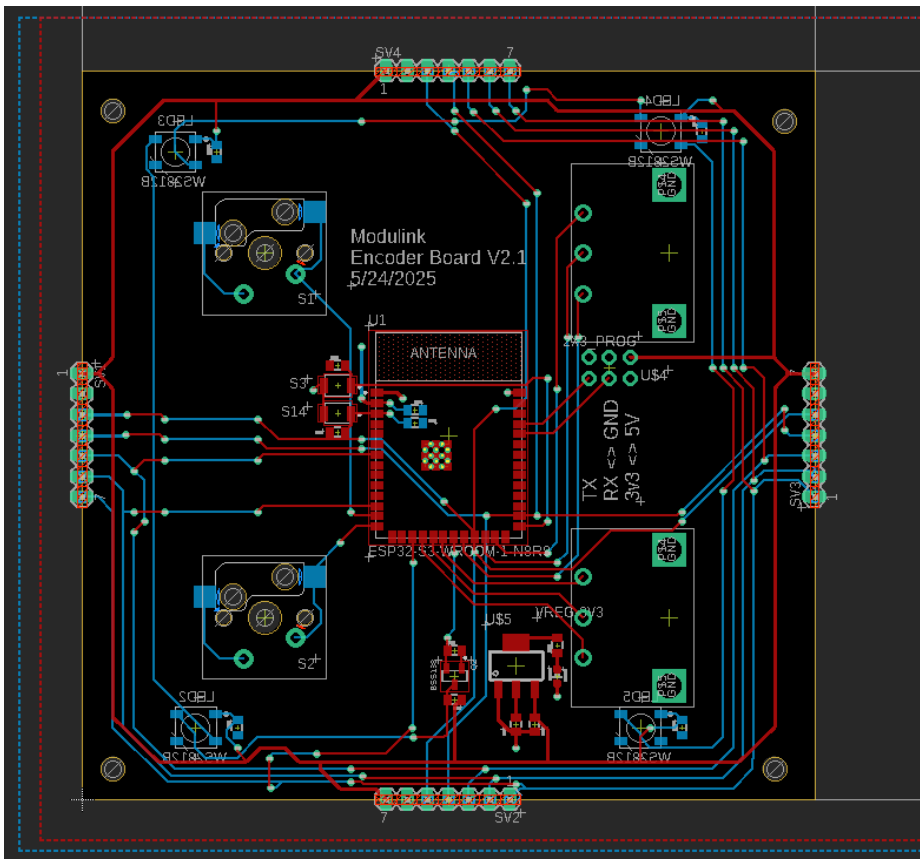

Figure 12: Encoder PCB

The above board is our Encoder Board, which also has a few key design decisions. Instead of placing the ESP32S3 at the edge of the board to ensure correct antenna function, the ESP32S3 is placed at the center instead. This was done because the encoder module is a peripheral device and thus will not be using the antenna. Therefore, the antenna was placed in the middle of the

board to minimize the I2C line distances to all 4 sides. Additionally, each input component was spaced out to be nearly as wide as possible to not over crowd regions and ensure the user will not be interacting with the wrong component.

Enclosure

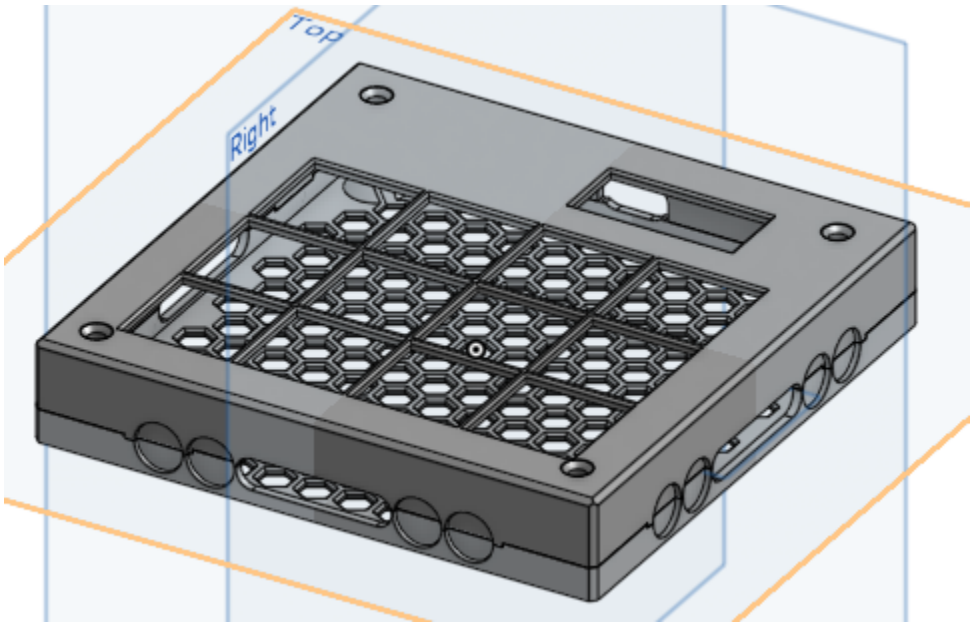

Figure 13: 3D model of the macropad enclosure

3D-printed enclosures were created for both the macropad module and the rotary encoder module. The design of these enclosures reflected the constraints imposed by the manufacturing process, as well as the need for the modules to be physically compatible with each other and their constituent components.

The enclosures were printed with 0.20 mm accuracy. Accounting for expansion, this could up to triple in size as the filament is being laid down. Due to this, and because past experience had shown that a clearance of less than 0.50 mm would lead to parts coming into too close contact with the filament, we added a 0.50 mm clearance to all potential points of collision between the enclosure and its inner components: namely, the PCB itself, the magnetic connectors on the sides

of the enclosure, the protruding antenna of the macropad board, and the keycaps/encoder stems. Each enclosure was manufactured in two halves split along the equator so that it could be assembled around the circuit board. The plane of splitting was chosen to be along the centerline of the magnetic connectors, so as to minimize the amount of support required when printing.

The two halves of each enclosure are held together by four bolt-nut pairs at the corners, which goes through corresponding holes in the PCB. Chamfered countersinks at the top and hexagonal insets at the bottom ensure that the bolts and nuts sit flush. The height of the enclosure, 17.5 mm, allows one to choose either 20 mm bolts for short legs or 25 mm bolts for longer legs.

All the enclosures were verified to have identical external dimensions, which was an easy way to verify that they would connect and fit when brought together. The main method by which different modules are physically connected to each other is by the magnets in the magnetic connectors. Additional slots for 8mm diameter magnets were added next to these connectors, but they are largely unnecessary.

To prevent users from experiencing discomfort when encountering the edges of the enclosure, a chamfer was added to all external edges except the bottom edges. In addition, a honeycomb pattern was cut out of the bottom of each enclosure to allow the LED underglow to be visible to the user.

Final Product

Initial Goal vs. Final Product

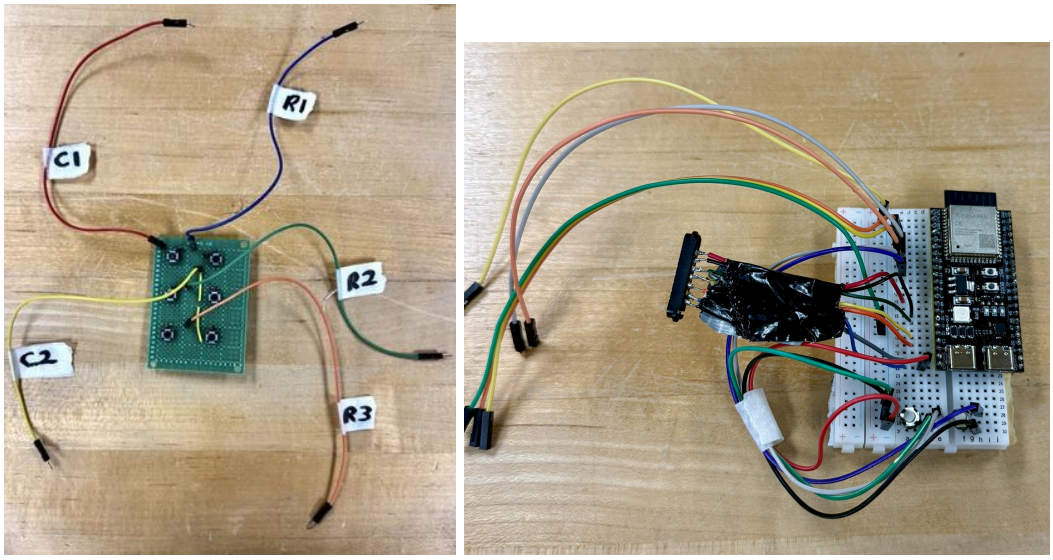

Figure 14: Macropad and encoder board prototypes

Each of the above prototypes demonstrated the core functionalities of each module, allowing testing of key presses, encoder spins, and website configurations.

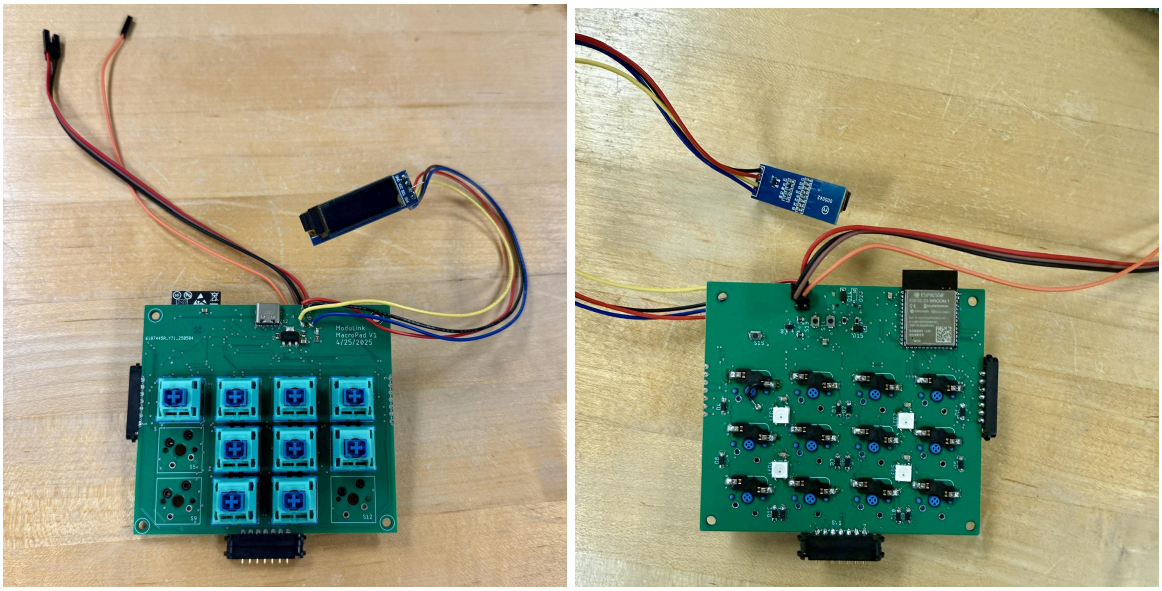

Figure 15: Macropad V1

The above figure is the first version of the macropad. This version demonstrated the main functionalities of keypresses, website-server hosting, and I2C communication. It was extensively to iterate firmware for the macropad board, allowing us to make the transition from standard firmware to RTOS firmware. There were 2 main issues that were addressed in the redesign. The first was the key switch spacing, as the key switches were placed too close together along the y axis causing the keycaps to catch on each other. The second issue was the board sizing, which was originally 8cm by 9cm. This differed from our agreed upon 9cm by 9cm standard in order to accommodate for the device antenna; however, it was later determined that the antenna could stick outside of the enclosure, and thus the board should follow the 9cm by 9cm standard.

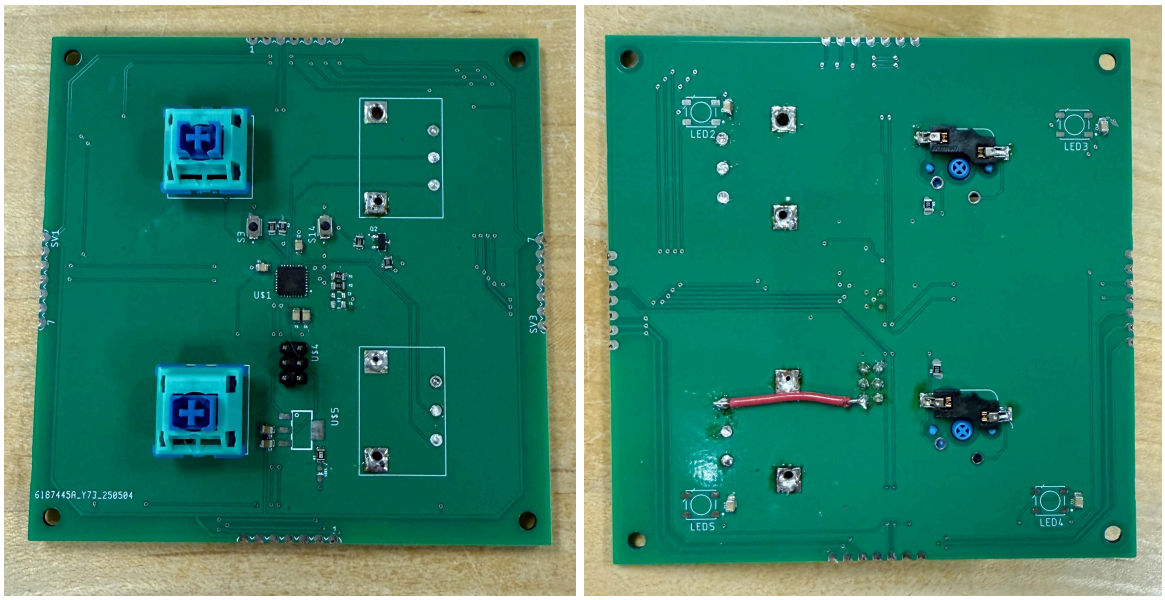

Figure 16: Encoder board V1

The above figure is the first version of the encoder board. Unlike the macropad version 1, this board did not achieve functionality, and much time was spent debugging and trying to flash code to the ESP32C3 chip on board.

Figure 17: Encoder board V2 and macropad board V2 (final version)

The image above shows the final versions of both the macropad and the encoder modules.

Macropad V1 vs. V2.

The main improvement from V1 to V2 was the adjustment in vertical key spacing. In V1, the spacing was too tight, causing the keycaps to rub against each other, which impacted usability. In V2, we increased the vertical spacing slightly, which not only resolved the issue but also allowed us to resize the PCB from 8 cm × 9 cm to 9 cm × 9 cm, making it match the footprint of the encoder modules.

Encoder V1 vs. V2:

The key change in the encoder module was the switch from an ESP32-C3 to an ESP32-S3. Due to time constraints and the complexity of working with the C3 chip, we decided to use the S3 despite it being overkill for the encoder's functionality. Since we already used the S3 for the macropad and had a working setup, it was a practical choice that guaranteed reliability within our timeframe.

Performance and Limitations

The main macropad has a key limitation: it can only support up to 9 modules due to constraints in the custom communication protocol we implemented. This is currently the most significant limitation of the system.

Another potential issue is power consumption. The current draw under various configurations is as follows:

Components	Power Draw
Macropad only	120 mA
Macropad + 1 encoder	175 mA
Macropad + 2 encoders	200 mA

Extrapolating from these values, a fully populated system with 9 modules could potentially exceed 2 A, which is far above the maximum 0.9 A typically available from a standard 5 V USB laptop port.

To address this, we will need to explore power optimization strategies, such as implementing low-power modes for inactive modules and other current-saving techniques, to ensure the system remains within USB power limits and operates reliably.

Challenges Encountered

As a group, we encountered many difficulties in the design and implementation of our product. The main challenges that we came across are as follows: ESP32C3 bootloading issues, website-server connection issues, and I2C and intermodule communication difficulties. The first major challenge that we faced came with the first version of the encoder board. For this version, we decided to switch from the ESP32S3 to an ESP32C3 due to less demanding software, lower pricing, and sizing considerations. In addition to featuring a QFN package, this chip uses the

RISC-V architecture and has different startup pin functionalities. After soldering the Encoder Board V1, we had considerable difficulties communicating with the chip. After fixing unintended shorts due to PCB design errors, we were unable to communicate with the chip via UART. We tested the current draw of the board and found that the entire board was only drawing 6mA. This seemed consistent with one of the many sleep states listed in the C3's datasheet, and since we were unable to determine a way to "wake the chip up", we decided to switch to the S3 for the second version

The second issue that we faced was difficulties connecting to the macropad's server when connected to eduroam. At first, we were unable to connect to eduroam due to the WPA-2/WPA-3 double input nature of eduroam. We found a fix for this issue by using the #PEAP branch of the WiFi manager library; however, even after we were able to successfully connect to eduroam through the WiFi manager, we were unable to connect to the device's server from our computers. We found that this was not an issue when working on non-WPA-2/WPA-3 wifi networks, and thus most of our website testing was done on home wifi networks or personal cellphone hotspots.

The final issue that we faced was with the I2C communication between modules. In our testing we received numerous "unexpected NACK detected" errors from the Wire.h library when transmitting data from the encoder module to the macropad module. Through testing, we determined that the errors came from Serial.print statements located within the encoder's I2C handler functions, which when removed eliminated the errors.

Planning and Organization

	Start Date	End Date	Timeline	Status
ESD2	Apr 1, 2025	Jun 13, 2025		
Milestone 1	Apr 1, 2025	Apr 11, 2025		Complete
Define project goals	Apr 1, 2025	Apr 7, 2025		Complete
Research components	Apr 8, 2025	Apr 11, 2025		Complete
Prepare order	Apr 10, 2025	Apr 11, 2025		Complete
Milestone 2	Apr 12, 2025	Apr 27, 2025		Complete
Test Macro pad with ESP32	Apr 12, 2025	Apr 16, 2025		Complete
Test out I2C with 2 modules	Apr 15, 2025	Apr 17, 2025		Complete
Build website to interface with modules	Apr 17, 2025	Apr 21, 2025		Complete
Milestone 3	Apr 28, 2025	May 11, 2025		Complete
Order Initial PCB prototype	Apr 28, 2025	Apr 30, 2025		Complete
Test firmware combined with backend	May 2, 2025	May 6, 2025		Complete
3d prinit inital hardware and aligning modules	May 6, 2025	May 11, 2025		Complete
Milestone 4	May 12, 2025	May 25, 2025		Complete
Look into other module types	May 12, 2025	May 14, 2025		Complete
Prepare order	May 14, 2025	May 15, 2025		Complete
Refine backend while waiting for parts	May 16, 2025	May 20, 2025		Complete
3d prinit revision with PCB	May 21, 2025	May 25, 2025		Complete
Milestone 5	May 26, 2025	Jun 8, 2025		Complete
Research improved code	May 26, 2025	May 28, 2025		Complete
Implement new code and firmware	May 28, 2025	Jun 1, 2025		Complete
Create final PCB	May 26, 2025	Jun 1, 2025		Complete
Milestone 6	Jun 2, 2025	Jun 13, 2025		Complete
Design 3D printed enclosure for both modules	Jun 1, 2025	Jun 8, 2025		Complete
Test and debug operation with multiple modules	Jun 2, 2025	Jun 9, 2025		Active
Prepare online project	Jun 2, 2025	Jun 11, 2025		Complete
Write report	Jun 9, 2025	Jun 13, 2025		Active
		Burndown		

Figure 18: Gantt chart

Central Module Macropad

Item	Package	Components	Quantity	Description	Manufacturer Product Number	Link	Unit price	Unit Price for 100 units	Unit Price for 1000 units
Capacitor 0805 0.1uF	0805	C1,C4,C5,C6,C7	5	KR72A104KL N8	08051C104KAZ4A	A-AVX/08051C10	$0.2200	$0.0800	$0.0560
Capacitor 0805 1uF	0805	C2,C3,C8	3	1AR71C105KU N	0805YC105KAT2A	X/0805YC105KA	$0.1300	$0.0510	$0.0360
Diode for Switch Matrix	SOD-123	,D5,D6,D7,D8,D9,I	12	NDARD 100V 200	SMMSD4148T1G	d-4331842_sig-C	$0.1400	$0.0780	$0.0492
USB C Connector		J1	1	USB2.0 TYP C 2-	USB4105-GF-A	products/detail/c	$0.7800	$0.5597	$0.5246
Power LED	603	LED1	1	Single Color LED-	KB EELP41.12-P1R2-36-3X4X-5-R18	R2-36-3X4X-5-R1	$0.3100	$0.0760	$0.0550
Underglow Light LED	SMD 5050	D2,LED3,LED4,LE	4	050 5x5mm SMD	COM-24837	rkFun/COM-2483	$0.7500	$0.7500	$0.7500
Logic Level Shifter		Q2	1	Ts SOT-23 N-CH	BSS138	i-Fairchild/BSS1	$0.2700	$0.0770	$0.0650
Ressitor 10K	0805	R2,R3,R7,R8	4	05 10kOhms 1.0	AT0805FRD0710KL	)/AT0805FRD07	$0.3600	$0.1770	$0.1210
Ressitor 330	0805	R4	1	ID 330Ohms 1/	R0805FR-07330RL	)/AR0805FR-073	$0.2800	$0.1400	$0.0930
Ressitor 5.1K	0805	R5,R6	2	- SMD 1/8watt 5.	CRCW08055K10FKEA	55K10FKEA?qs=s	$0.1000	$0.0180	$0.0170
Switches		,S5,S6,S7,S8,S9,S	12	herry Americas LL	MX1A-11NN	d-91136_sig-Cj0K	$1.2500	$1.2500	$1.2500
Buttons		S13,S14	2	Top Actuated w/o	B3U-1000P	Electronics/B3U-	$0.9800	$0.6320	$0.5890
VREG		U2	1	INEAR 3.3V 1A S	296-TLV1117LV33DCYTDKR-ND	5604607097_dsa	$1.1100	$0.6395	$0.5489
ESP32		U1	1	MODULE WIFI PC	ESP32-S3-WROOM-1-N8	tail/espressif-sys	$5.4900	$4.1447	$3.5924
TVS Diode for VBUS	SOD123			DE 5VWM 9.2VC	SMF5.0A	roducts/detail/lit	$0.2800	$0.0990	$0.0980
TVS Diode for DATA	SOD123			E 3.3VWM 6.8VC	SMF3.3	products/detail/li	$0.3510	$0.1970	$0.1670
Enclosure 3D print		PETG	51g		Bambu Lab PETG HF	bambulab.com/p	$1.0200	$0.9200	$0.9200
PCB							$5.4000	$5.4000	$5.4000
2 pair Magnetic Connector			3	Magnetic Conne		BcFhFVzHnkBDg	$10.00	$10.00	$10.00
Custom Connector PCB			3				$0.32	$0.32	$0.32
Pogo pins for connector			21			85cfaa0a-fe17-49	1.53	1.53	1.53
OLED screen			1		Monochrome 0.91" 128x32 I2C OLED Display	-Luminous-Comp	2.39	2.39	2.39

Figure 19: Complete BOM for macropad module

Encoder Module									
General Desciption	Package Size	Board Component	Quantity	Description	Part Number	Link	Unit price for 1 unit	Unit price for 10 units	Unit price for 100 units
0.1 uF Capacitor	0805	2, C9, C10, C11, C1	4	21KR72A104KL NE	08051C104KAZ4A	A-AVX/08051C104	0.22	0.08	0.056
1 uF Capacitor	0805	C1, C7, C8	3	21AR71C105KU NE	0805YC105KAT2A	X/0805YC105KAT	0.13	0.051	0.036
Blue LED	603	LED1	1	Single Color LEDs	KB EELP41.12-P1R2-36-3X4X-5-R18	1R2-36-3X4X-5-R18	$0.310	$0.076	$0.055
RGB LED	SMD 5050	ED2,LED3,LED4,LED	4	5050 5x5mm SMD	COM-24837	arkFun/COM-24837	0.75	0.75	0.75
Mosfet		Q2	1	ETs SOT-23 N-CH	BSS138	ni-Fairchild/BSS138	$0.27	$0.077	$0.065
10 kOhm Resistor	0805	R2,R3,R4,R5,R6, R8,	8	805 10kOhms 1.0%	AT0805FRD0710KL	O/AT0805FRD0710	$0.36	$0.177	$0.121
330 Ohm Resistor	0805	R7	1	MD 330Ohmns 1/8	R0805FR-07330RL	O/AR0805FR-07330	$0.28	$0.14	$0.093
Keyboard Switches		S3,S14	2	Top Actuated w/o	B3U-1000P	rd-91136_sig-CjOK	$0.98	$0.632	$0.589
3.3V Voltage Regulator		U5	1	LINEAR 3.3V 1A SO	296-TLV1117LV33DCYTDKR-ND	5604607097_dsa-	$1.11	$0.640	$0.549
Buttons		S1, S2	2	Top Actuated w/o	B3U-1000P	-Electronics/B3U-1	$0.98	$0.632	$0.589
ESP32S3		U1	1	MODULE WIFI PCI	ESP32-S3-WROOM-1-N8	etail/espressif-syst	$5.4900	$4.1447	$3.5924
Encoder		U2, U3	2	ti-directional Enco	PEC16-4220F-N0024-ND	cts/detail/bourns-	1.69	1.356	1.0908
PCB							$5.4000	$5.4000	$5.4000
2 pair Magnetic Connector			4	n Magnetic Conne		bBcFhFVzHnkBDp	$10.00	$10.00	$10.00
Custom Connector PCB			4				$0.32	$0.32	$0.32
Pogo pins for connector			28			-85cfaa0a-fe17-49	1.53	1.53	1.53
Keycaps			2	bs for Keyboard Sw	B0D1DMLBG4	p/B0D1DMLBG4?s	0.599	5.99	59.9
Knobs			2	bs for Rotary Enco	B07F2768F5	-Insert-Potentiome	0.866	8.66	86.6

Figure 20: Complete BOM for encoder module

We used a group chat with all team members to coordinate meetings and collaborate on the project. Meetings were held in person when focused on hardware development, firmware testing, or code debugging, while Zoom was used for sessions dedicated to documentation, planning, and project management tasks.

To ensure efficiency and accountability, we divided the workload so that each member had ownership and responsibility over specific aspects of the project.

Andre was tasked with designing and 3D-printing the enclosures and production jigs, playing a key role in the physical realization of the device. In addition, he led research into the NRF52840 chip (an industry-standard microcontroller for wireless keyboards) that we plan to integrate in version 2 of the device. Andre was also responsible for exploring and implementing new features for the upcoming version, such as Bluetooth connectivity (supporting HID over Bluetooth) and battery integration.

Akila was responsible for the development of the macro pad, including prototyping, testing, PCB design, and firmware development for the central module. Additionally, he led the design of the firmware architecture and the website, ensuring effective integration between the hardware and the web-based interface.

Zach was assigned to the encoder module, overseeing component selection, prototyping, PCB design, firmware testing, and manufacturing. His efforts were critical to delivering a robust and reliable encoder module that functions seamlessly within the overall system.

Market Research

What you learned from interviews of experts

During our market research, we found that our product of a modular HID device was a relatively empty niche, and thus we did not find experts who had designed products like this before. Thus, we redefined experts as users who have a particular usage of the device in mind. From our interviews with these experts, we found that 2 priorities for our device were size and cost. Especially for digital art purposes, desk space is often at a premium, and smaller modules with fewer buttons were often preferable over large modules with excess functionalities. Additionally, due to the nature of HID devices often being redundant to a standard computer keyboard, overly high prices above 20-30 dollars per set would be a major downside.

What you learned from interviews of non-expert

From our non-expert interviews, being individuals who didn't thave a clear purpose for our device in mind, we mostly gathered information regarding module types and wired vs wireless capabilities. For module types, we learned that there were a large variety of different functionalities that potential users were interested, including touchpads, screens, and even foot pedals. Non-expert users were also pretty divided between wired vs wireless functionality, with some preferring the lack of charging required for wired devices and others preferring the less cluttered aspect of the wireless version.

Conclusion

What I learned. Throughout this course, I learned a lot about human interface devices, such as how the HID protocol works, how keyboard matrix scanning works, and how a segmented rotary encoder sends signals about how it's being turned. I realized that Arduino was a lot more capable than I originally gave it credit for, as it ended up being the framework that we used for all on-board programming. I also gained a lot more experience with CAD software, and learned some really neat tricks for making a cleaner CAD document. For example, I learned that you could make linear patterns of sketches and features, which was extremely helpful when making the cutouts for the key matrix and the honeycomb bottom. I learned about the "split" tool, which was really useful for iterating different planes at which to cut the case into two halves, and the "boolean" and "thicken" tool, which were useful for creating a perfectly shaped hole to accommodate another part while still providing some clearance. Finally, I learned to be more deliberate about the references I make in each feature, so that changing some feature early on in the process wouldn't destroy the entire document: very useful as I went through many iterations of this encoder.

What I would do differently if I could start over. If I could start over, I would make it the goal from the very beginning to have the device work, on battery, wirelessly over only a Bluetooth connection. I feel like that is much more realistic and convenient for a human interface device, and if we had aimed for that from the beginning, then it would have been a feasible goal. I feel like with the process we had, where we were independently working on the two prototypes, meant that much of the work and debugging that went into one of the development timelines wasn't applicable to the other. In addition, if I could start over, I think we should have done more market research at the beginning to see what actual use cases people would have and design

around those. We had simple applications such as keyboard shortcuts and volume control, but it would have been nice to get the perspective of someone who e.g. does animation and would use a rotary encoder to control the size of the pen.

Possible next steps. If we had more time, we would have worked on going fully wireless as well as adding additional modules.

For a wireless design, we would need to implement Bluetooth functionality, and were planning on switching from the ESP32S3 to the nRF52840 for that. The functionality for implementing an HID protocol-speaking device on the nRF was already done (the equivalent functionality on the existing prototype would've been the Arduino USBHIDKeyboard library), we'd just need to hook it up to the keyboard scanning matrix. We would also need to add a battery to the central module, or add a separate battery module entirely.

For additional modules, we have several ideas from our market research interviews of functionality that people thought would have been useful, and which could have become additional modules. For example, many interviewees expressed interest in a touchpad module for reading gestures and sending commands based on those; this would have been useful for e.g. digital art or image editing applications. Another "high-demand" feature was a screen module. Our current macropad module has a screen, but it is very small and can only display a single string. With additional time, the screen module could be made much larger, and capable of displaying connectivity status, battery status, etc.

Class Feedback

Learning. Yes, I would say this class taught me about as much as I expected to learn going in. Since it was just a continuation of EE 326, I knew that I wouldn't be learning that many new

skills, but I was happy to be able to develop the skills that I had learned last quarter: especially CAD.

Suggestions for improvement. No, I think this class was structured and paced very well for learning. The fact that we had to make a Gantt chart at the beginning forced us to at least have a plan and a timeline, even if it was flexible. I think the pacing of the milestones and when prototypes were expected was very fair and reflected the actual design process, at least for our design. It was unfortunate that classmates weren't able to be as helpful this quarter, because everyone had different projects, but that's not something that can be changed.

MCU. I think the ESP32 was a fine choice as a default microcontroller, because it's capable of doing most tasks fairly well. I would (obviously) have preferred to work with an nRF microcontroller due to it having lower power consumption. Perhaps if it was a requirement to make a project that is battery powered, and the requirement for having a Wi-Fi website was removed, the nRF would make a good default choice.

Overall thoughts. I definitely enjoyed this course. I must admit I didn't put in as much time into this quarter as I was able to last quarter, and that reflected negatively on what I was able to get out of it. This is especially true since I wasn't able to finish the second prototype! However, I am still proud of what we have been able to accomplish, and I plan on finishing the prototype in my free time during the summer. My favorite part of this project was unironically when things randomly stopped working that had just worked hours/days ago, because it was a stark reminder about the difficulties of working with delicate electronics, and it was nice to vent with the team about it.